

Answer the Questions

Question 1: Why is it recommended to explicitly assign values to enum members in some cases?

- Explicitly assigning values to enum members is recommended when the values need to remain **fixed and consistent**, especially when they are stored in a database, used by APIs, or represent specific values such as **Flags**. This prevents accidental changes to their numeric values.

Question 2: What happens if you assign a value to an enum member that exceeds the underlying type's range?

- It causes a **compile-time error**, because the assigned value is outside the valid range of the enum's underlying type.

Question 3: What is the purpose of the virtual keyword when used with properties?

- The `virtual` keyword allows a property to be **overridden in a derived class**, enabling runtime polymorphism and providing different property behavior in the derived class.

Question 4: Why can't you override a sealed property or method?

- Because the `sealed` keyword prevents further overriding of a property or method in derived classes. It is used to **stop inheritance-based modification** of that member.

Question 5: What is the key difference between static and object members?

- **Static members** belong to the class itself and are shared by all objects, while **object (instance) members** belong to individual objects, so each object has its own copy.

Question 6: Can you overload all operators in C#? Explain why or why not.

- No. C# allows overloading **most operators**, but some operators such as `&&`, `||`, `=`, `..`, and `?:` **cannot be overloaded** because their behavior is defined by the language.

Question 7: Can you overload all operators in C#? Explain why or why not.

- No. C# allows overloading **most operators**, but some operators such as `&&`, `||`, `=`, `..`, and `?:` **cannot be overloaded** because their behavior is defined by the language.

Question 8: When should you consider changing the underlying type of an enum?

- You should consider changing the underlying type when you need to **save memory**, ensure **compatibility with external systems or databases**, or support a specific range of values.

Question 9: Why can't a static class have instance constructors?

- Because a static class **cannot be instantiated**, so it has no objects or instances that require an instance constructor.

Question 10: What is the difference between overriding Equals and == for object comparison in C# struct and class ?

- `Equals()` is used to determine **logical/value equality** and can be overridden to compare the actual data of objects.
- `==` is a n operator. For **classes**, it normally compares references unless overloaded; for **structs**, it compares values when supported.
- For custom classes, overriding `Equals()` does **not automatically change ==**; you must overload `==` separately.

Question 11: Why is overriding ToString beneficial when working with custom classes?

- Overriding ToString() provides a **meaningful string representation** of an object instead of the default class name, making objects easier to **display, debug, and log**.

Question 12: Can generics be constrained to specific types in C#? Provide an example.

- Yes. Generics can be constrained using the where keyword to allow only specific types.

```
</> C#  
  
class Repository<T> where T : class  
{  
    // T must be a reference type  
}
```

Question 13: What are the key differences between generic methods and generic classes?

- **Generic Class:** The type parameter applies to the **entire class** and its members.
- **Generic Method:** The type parameter applies only to the **specific method** that uses it.

```
</> C#  
  
class Box<T> { } // Generic class  
  
void Print<T>(T value) // Generic method  
{  
}  
}
```

Question 14: Why might using a generic swap method be preferable to implementing custom methods for each type?

- A generic Swap method can work with **different data types using the same code**, reducing code duplication and improving **reusability and type safety**.

Question 15: How can overriding Equals for the Department class improve the accuracy of searches?

- Overriding Equals() allows the Department class to compare objects based on their **meaningful data** instead of their references, making searches more **accurate and reliable**.

Question 16: Why is == not implemented by default for structs?

- Because C# does not automatically define == for custom structs. The operator must be **explicitly overloaded** if you want to compare their values using ==.